

## **Design Proposal**

### **LLM-Powered Academic Research and Information-Gathering Planning Agent**

#### **1.0 System Requirements and Assumptions**

The proposed system is an LLM-powered planning agent for academic research and information gathering. It receives a high-level research topic, decomposes it into searchable sub-questions, retrieves scholarly records from online academic services, filters, ranks and summarises the evidence, validates traceability, and saves the approved research package offline.

- Structured academic metadata APIs: Crossref as the primary open source, OpenAlex as a complementary scholarly graph (free API key required), and Semantic Scholar as an optional third source subject to API key availability (Crossref, 2026; OpenAlex, 2026; Semantic Scholar, 2026).
- An instruction-tuned LLM accessed via Hugging Face Inference Providers for planning and summarisation, with a locally runnable model as a fallback given limited free-tier credits (Hugging Face, 2026).
- Orchestration in Python 3.11+ using LangGraph, with SQLite for prototype state and a minimal FastAPI or Streamlit interface.
- Offline output as human-readable Markdown alongside machine-readable JSON/CSV, with source citations and run metadata.
- A single mandatory human-approval point before any consequential action, such as final export.

## 2.0 Design Decisions

- **Layered hybrid architecture** over purely deliberative or purely reactive designs (Wooldridge, 2009). A deliberative agent alone cannot revise its plan as fast as the environment invalidates it – a dead source, an empty result set – while a purely reactive agent (Brooks, 1991) responds immediately but holds no persistent goal, so it cannot judge when evidence is sufficient. Because the task is goal-directed yet source availability and result quality are only discoverable at runtime, both are required.
- **Two-pass vertical layering** – context passed upward, control returned downward – over horizontal layering, which would require arbitration between six agents proposing actions independently on one observation. Escalation, in the manner of InteRRaP (Müller and Pischel, 1993), was preferred over the suppression-based conflict resolution of TouringMachines (Ferguson, 1992): an unresolved problem is passed to the layer above rather than layers competing to produce one output.
- **LangGraph** over CrewAI and Microsoft Agent Framework. Its explicit graph and state model gives finer-grained control over the validation and bounded-retry paths this design requires than CrewAI’s higher-level abstractions, and it is more established than the newer Microsoft framework (CrewAI, 2026; LangChain, 2026; Microsoft, 2026a). AutoGen was excluded: its repository is in maintenance mode and directs new users to Microsoft Agent Framework (Microsoft, 2026b; Wu et al., 2024).
- **Structured API querying** of Crossref and OpenAlex over vector-embedding search or unrestricted scraping, for provenance, testability and citation validation.

### **3.0 Approach and Methodology**

#### **3.1 Development methodology**

- Scrum governs team organisation and incremental delivery but supplies no agent-oriented modelling vocabulary.
- An adapted Prometheus methodology (Padgham and Winikoff, 2003; 2004) supplies this, preferred over Gaia (Wooldridge, Jennings and Kinny, 2000) because it reaches implementation-facing artefacts rather than stopping at abstract role models; Figures 1 and 2 correspond to its system overview and interaction protocol artefacts.

#### **3.2 Agent-oriented techniques**

- The Prometheus specification produced six cohesive, loosely coupled roles: Orchestrator/Supervisor, Planner, Academic Retrieval, Processing/Ranking, Evidence Validator and Storage/Export (Figure 1).
- The Orchestrator/Supervisor and Planner are deliberately separated: the Planner decomposes goals into subtasks, while the Orchestrator/Supervisor maintains execution state, routing and retries.
- LangGraph, the model gateway and the shared state store are modelled as supporting components rather than autonomous agents.
- BDI (Bratman, Israel and Pollack, 1988) is a conceptual influence rather than an implementation: Prometheus assumes a design-time plan library, whereas subtask plans here are generated at runtime.

### 3.3 Planning and execution approach

- Plan-and-Execute governs the outer loop, chosen over a single end-to-end ReAct loop because it yields an inspectable subtask sequence before retrieval begins; ReAct (Yao et al., 2023) then governs reasoning and acting within each subtask.
- On failed validation, the Evidence Validator requests a bounded, targeted retry of retrieval or processing, routed by the Orchestrator/Supervisor; escalation to the Planner for re-planning follows only once retries are exhausted (Figure 2).
- Autonomy is bounded: consequential actions require human approval before final export, as specified in the system requirements.

### 4.0 Anticipated Challenges

- Error propagation across specialised agents: because the architecture has sequential dependencies, an early failure — for example, the Planner mis-decomposing an ambiguous request — can degrade retrieval, processing and validation downstream.
  - Mitigation: the Evidence Validator returns failed checks for targeted retrieval or reprocessing, with defined thresholds, retry limits and termination conditions to prevent repeated error cycles.
- Traceability and shared-state consistency: evidence, identifiers, summaries, validation decisions and outputs must remain correctly associated as they move between agents; inconsistent state management risks duplicated evidence, stale intermediate results, or claims that cannot be traced to their source.

- Mitigation: persistent checkpoints, structured inter-agent message schemas, versioned outputs and audit logs, at the cost of added implementation complexity.
- Explainability of LLM-based decisions: the reasoning behind decomposition, ranking and summarisation choices is not directly observable, weakening user trust and making outputs hard to trace to a justification.
  - Mitigation is structural rather than post hoc: Plan-and-Execute exposes an inspectable subtask sequence before execution, ReAct traces record interleaved reasoning and actions within each subtask (Yao et al., 2023), and the human approval point places a reviewable plan and evidence package before the user prior to export. Generated rationales may nonetheless not faithfully reflect the model's actual computation.

## 5.0 Graphical System Designs

Two complementary views are provided: a system overview showing agent roles and control flow, and a UML sequence diagram showing message exchange over time, including the bounded-retry and escalation paths.

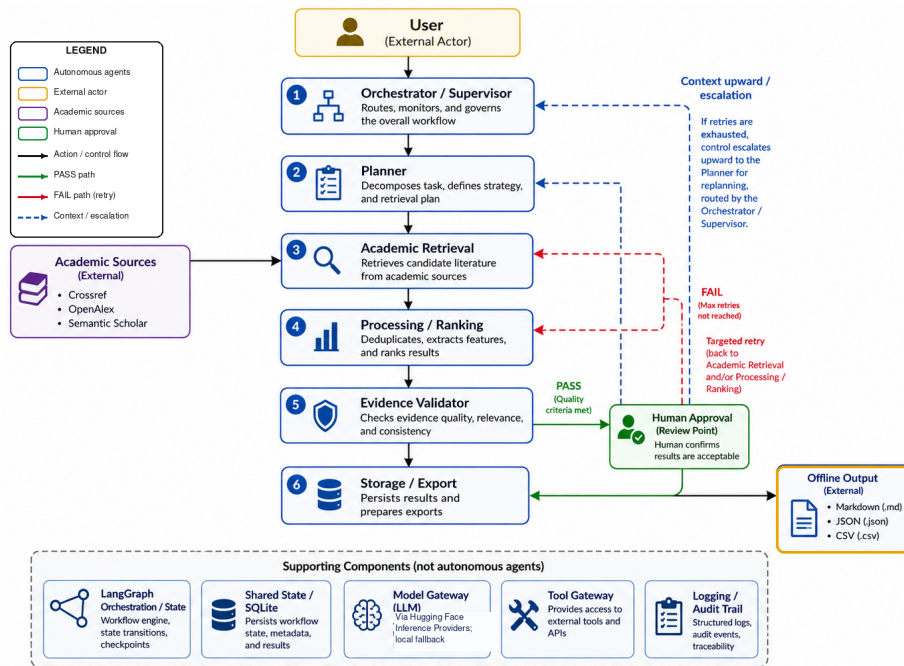


Figure 1. System overview of the proposed six-agent academic research and information-gathering system.

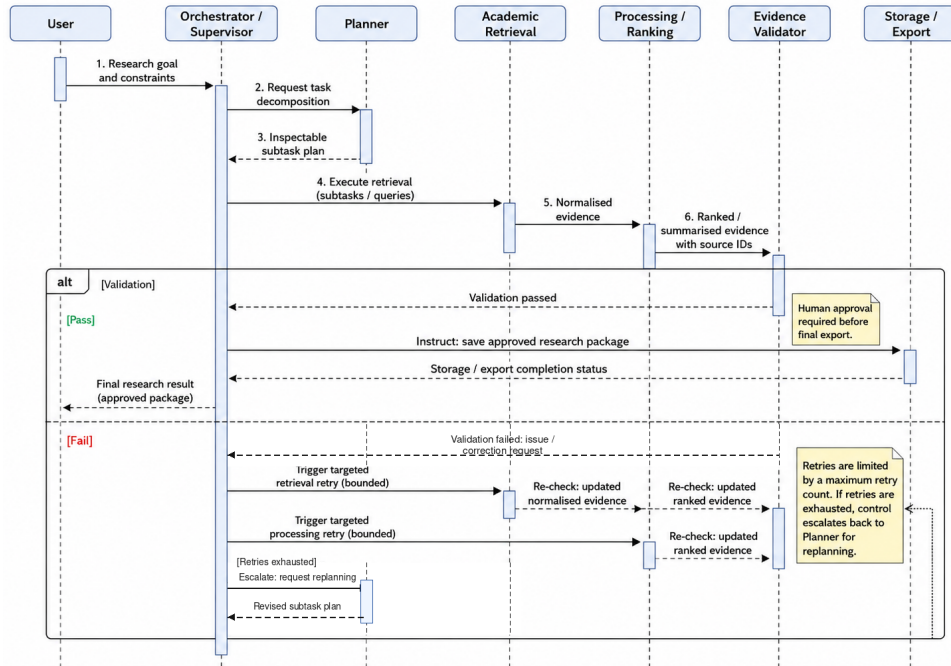


Figure 2. UML sequence diagram for an academic research request, including bounded retry and escalation.

## 6.0 Tools, Libraries, Models and Environments

- Orchestration and language: Python 3.11+ with LangGraph.
- Interface: FastAPI with a minimal web interface, or Streamlit for the first demonstration.
- State and persistence: SQLite for the prototype; PostgreSQL if concurrent or production use is required.
- Academic retrieval: Crossref REST API and OpenAlex as primary sources; Semantic Scholar as optional.
- LLM access: Hugging Face Inference Providers, with a locally runnable model as fallback.
- Data validation: Pydantic models for every inter-agent message.
- Observability: structured logs, trace IDs, and latency/token/cost metrics.

- Testing and evaluation: an evaluation dataset exceeding the initial 10–20 research requests, covering varying scope and ambiguous topics, against a target of at least 90% error-free execution.

## 7.0 References

- Bratman, M.E., Israel, D.J. and Pollack, M.E. (1988) 'Plans and resource-bounded practical reasoning', *Computational Intelligence*, 4(3), pp. 349–355.
- Brooks, R.A. (1991) 'Intelligence without representation', *Artificial Intelligence*, 47(1–3), pp. 139–159.
- CrewAI (2026) CrewAI documentation. Available at: <https://docs.crewai.com/> (Accessed: 8 August 2026).
- Crossref (2026) REST API: metadata retrieval. Available at: <https://www.crossref.org/documentation/retrieve-metadata/rest-api/> (Accessed: 8 August 2026).
- Ferguson, I.A. (1992) *TouringMachines: an architecture for dynamic, rational, mobile agents*. PhD thesis. University of Cambridge.
- Hugging Face (2026) Inference Providers. Available at: <https://huggingface.co/docs/inference-providers/en/index> (Accessed: 8 August 2026).
- LangChain (2026) LangGraph overview. Available at: <https://docs.langchain.com/oss/python/langgraph/overview> (Accessed: 8 August 2026).
- Microsoft (2026a) Microsoft Agent Framework overview. Available at: <https://learn.microsoft.com/en-us/agent-framework/overview/> (Accessed: 8 August 2026).
- Microsoft (2026b) AutoGen repository. Available at: <https://github.com/microsoft/autogen> (Accessed: 8 August 2026).

Müller, J.P. and Pischel, M. (1993) The agent architecture InteRRaP: concept and application. Research Report RR-93-26. Saarbrücken: German Research Center for Artificial Intelligence (DFKI).

OpenAlex (2026) API reference: introduction. Available at: <https://developers.openalex.org/api-reference/introduction> (Accessed: 8 August 2026).

Padgham, L. and Winikoff, M. (2003) 'Prometheus: a methodology for developing intelligent agents', in Giunchiglia, F., Odell, J. and Weiß, G. (eds.) Agent-Oriented Software Engineering III. Berlin: Springer, pp. 174–185.  
doi:10.1007/3-540-36540-0\_14.

Padgham, L. and Winikoff, M. (2004) Developing Intelligent Agent Systems: A Practical Guide. Chichester: John Wiley & Sons.

Semantic Scholar (2026) Academic Graph API. Available at: <https://api.semanticscholar.org/api-docs/> (Accessed: 8 August 2026).

Wooldridge, M.J. (2009) An Introduction to Multiagent Systems. 2nd edn. Chichester: John Wiley & Sons.

Wooldridge, M., Jennings, N.R. and Kinny, D. (2000) 'The Gaia methodology for agent-oriented analysis and design', *Autonomous Agents and Multi-Agent Systems*, 3(3), pp. 285–312.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D. and Wang, C. (2024) 'AutoGen: enabling next-gen LLM applications via multi-agent conversation', *First Conference on Language Modeling (COLM 2024)*, Philadelphia, PA, 7–9 October.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023)  
'ReAct: synergizing reasoning and acting in language models', Eleventh  
International Conference on Learning Representations (ICLR 2023). Kigali,  
Rwanda, 1–5 May. Available at: <https://arxiv.org/abs/2210.03629> (Accessed:  
13 August 2026).